Git Hooks dan Automation dengan Golang

Tutorial dari nol: membangun hook pre-commit, commit-msg, dan pre-push dengan satu program Go

> **Yang Anda butuhkan**
> Go 1.22 atau lebih baru, Git, editor teks, dan terminal (Windows: Git Bash). Perkiraan waktu 60 sampai 90 menit. Tidak perlu akun GitHub kecuali untuk Tahap 9.

Cara memakai tutorial ini

Mulailah dari bab Pengenalan untuk memahami konsepnya. Setelah itu kerjakan tahap-tahapnya berurutan. Setiap tahap berisi kode mentah untuk dibuat dengan editor Anda, dan kotak hijau **CEK** di akhir tahap untuk memastikan Anda tidak salah jalan sebelum lanjut. Jika hasil cek berbeda, jangan lanjut dulu: lihat bagian Pemecahan masalah di akhir.

Yang akan Anda bangun

Satu program Go bernama `githook` yang dipanggil Git pada tiga momen: sebelum commit (memeriksa format dengan gofmt dan go vet), saat pesan commit ditulis (memeriksa format Conventional Commits), dan sebelum push (menjalankan test). Hook disimpan di folder `.githooks` yang ikut di-commit, sehingga satu tim memakai aturan yang sama.

Daftar tahap

Pengenalan. Apa itu Git hooks dan kenapa Go

Tahap 0. Persiapan

Tahap 1. Membuat proyek dan repo Git

Tahap 2. Kode contoh dan test

Tahap 3. Kerangka program githook dan perintah install

Tahap 4. Hook pre-commit: format dan go vet

Tahap 5. Hook commit-msg: Conventional Commits

Tahap 6. Hook pre-push: jalankan test

Tahap 7. Uji end-to-end dengan Git sungguhan

Tahap 8. Opsional: pakai Lefthook untuk tim

Tahap 9. Mengulang pemeriksaan di CI (GitHub Actions)

Pemecahan masalah dan langkah berikutnya

Pengenalan: apa itu Git hooks?

Tujuan: memahami konsepnya dulu sebelum menulis kode. Bagian ini cukup dibaca, tidak ada yang perlu diketik.

Masalah yang ingin diselesaikan

Dalam kerja tim, dan juga dalam tugas kelompok, tiga hal sering merusak kualitas kode: **format kode yang berbeda-beda**, **pesan commit asal-asalan** ("update", "fix lagi"), dan **test yang gagal** yang baru ketahuan setelah kode dikirim. Ketiganya bisa diperiksa oleh mesin, tetapi manusia sering lupa. Git hooks memindahkan pemeriksaan itu ke momen yang paling murah: sebelum commit dan sebelum push, saat kesalahan masih mudah diperbaiki.

Apa itu Git hooks

Hook adalah **program atau skrip yang dijalankan Git secara otomatis pada titik tertentu**, misalnya sebelum commit dibuat. Hook disimpan di sebuah folder hooks dan hanya dijalankan jika berkas hook-nya memiliki izin executable. Isi hook sepenuhnya kita yang menentukan: bisa shell script, Python, atau program Go seperti di tutorial ini.

Cara kerja: kode keluar menentukan nasib aksi Git

Git menjalankan hook lalu melihat **kode keluar** (exit code) program tersebut. **Kode 0 berarti lanjut. Selain 0 berarti Git membatalkan aksinya.** Itu satu-satunya aturan yang perlu diingat. Alur yang akan kita bangun:

```
git commit
  -> pre-commit   cek format (gofmt) dan go vet     gagal = commit batal
  -> commit-msg   cek format pesan commit           gagal = commit batal
  -> commit tercatat

git push
  -> pre-push     jalankan semua test                gagal = push batal
```

Hook yang paling sering dipakai

Hook	Kapan berjalan	Contoh pemakaian
pre-commit	Sebelum commit dibuat	Cek format, lint, go vet
commit-msg	Setelah pesan commit ditulis	Cek format pesan commit
pre-push	Sebelum push ke remote	Menjalankan test
post-merge	Setelah merge atau git pull	Instal dependensi, regenerate kode

Tutorial ini membangun tiga yang pertama. Git memiliki hook lain, termasuk yang berjalan di sisi server; daftar lengkapnya ada di dokumentasi resmi (git-scm.com/docs/githooks).

Di mana hook disimpan

Secara bawaan hook berada di `.git/hooks`, tetapi folder `.git` **tidak ikut di-commit**, jadi hook di sana tidak sampai ke rekan satu tim. Solusinya: simpan hook di folder biasa (kita memakai nama `.githooks`) lalu arahkan Git ke sana dengan `git config core.hooksPath .githooks`. Pengaturan ini tersimpan di konfigurasi lokal, jadi setiap orang tetap perlu menjalankannya sekali. Karena itu tutorial ini membuat perintah `install` agar pemasangannya satu perintah saja.

Kenapa memakai Go untuk menulis hook

Hook boleh ditulis dalam bahasa apa pun. Go dipilih karena hasil build-nya **satu berkas** tanpa runtime tambahan, **berjalan sama di Windows, macOS, dan Linux** (shell script sering bermasalah di Windows), **cepat**, dan logikanya **bisa diuji** seperti kode biasa. Alat populer seperti Lefthook juga ditulis dengan Go. Catatan jujur: demi sederhana, tutorial ini memanggil program lewat `go run`; untuk tim sungguhan, build sekali menjadi binary agar hook lebih cepat.

Conventional Commits dalam satu menit

Hook `commit-msg` kita akan memeriksa format pesan commit dari spesifikasi Conventional Commits: **type(scope opsional): deskripsi**. Format yang konsisten membuat riwayat mudah dibaca dan memungkinkan nomor versi serta changelog dibuat otomatis (fix berkaitan dengan kenaikan PATCH, feat dengan MINOR, dan perubahan yang merusak kompatibilitas dengan MAJOR).

```
feat(auth): tambah login dengan Google
fix(cart): hitung total dengan diskon
docs: perbarui README
feat(api)!: ubah format respons       <- tanda ! = perubahan yang merusak
```

Batasan yang perlu diketahui sejak awal

Hook lokal adalah **alat kenyamanan, bukan penjaga keamanan**: siapa pun bisa melewatinya dengan `git commit --no-verify`, dan hook tidak otomatis terpasang saat repo di-clone. Karena itu pemeriksaan yang sama sebaiknya diulang di CI (Tahap 9). Hook memberi umpan balik cepat; CI menjamin aturan benar-benar ditegakkan.

Istilah yang dipakai di tutorial

Istilah	Arti singkat
staging	Area tempat file menunggu di-commit, diisi dengan `git add`
kode keluar	Angka yang dikembalikan sebuah program; 0 = sukses, selain 0 = gagal
shim	Skrip pendek di `.githooks` yang hanya memanggil program Go kita
gofmt / go vet	Alat bawaan Go untuk merapikan format dan mendeteksi kesalahan umum
CI	Server yang menjalankan pemeriksaan otomatis setiap kali kode dikirim

Tahap 0. Persiapan

Tujuan: Memastikan Go dan Git siap dipakai.

Buka terminal. Di Windows pakai **Git Bash** (ikut terpasang bersama Git for Windows), bukan CMD, karena hook kita memakai shell `sh`.

Pasang **Go 1.22 atau lebih baru** (go.dev/dl) dan **Git** (git-scm.com) jika belum ada. Lalu cek versinya:

Terminal

```
go version
git --version
```

Jika Git belum tahu siapa Anda, atur identitas sekali saja (lewati bila sudah pernah):

Terminal

```
git config --global user.name "Nama Anda"
git config --global user.email "email@anda.com"
```

> **CEK** Kedua perintah versi harus menampilkan angka versi.
>
> `go version`
>
> **Hasil yang benar:** go version go1.22 ... (atau lebih baru)

Tahap 1. Membuat proyek dan repo Git

Tujuan: Folder kerja, repo Git, dan modul Go.

Buat folder proyek, jadikan repo Git, lalu buat modul Go. Nama modul boleh bebas; di tutorial ini `githooks-demo`.

Terminal

```
mkdir githooks-demo
cd githooks-demo
git init -b main
go mod init githooks-demo
mkdir -p cmd/githook internal/calc
```

> **CEK** Struktur awal sudah terbentuk.
>
> `ls`
> `cat go.mod`
>
> **Hasil yang benar:** ls menampilkan: cmd go.mod internal
> go.mod berisi baris `module githooks-demo` dan baris `go 1.xx`

Tahap 2. Kode contoh dan test

Tujuan: Sesuatu yang bisa diperiksa dan diuji oleh hook.

Hook butuh bahan untuk diperiksa. Buat paket kecil `calc` beserta satu test. Buat dua berkas berikut dengan editor Anda.

Berkas: internal/calc/calc.go

```
// Package calc hanyalah kode contoh agar hook punya sesuatu untuk diperiksa.
package calc

// Add menjumlahkan dua bilangan.
func Add(a, b int) int {
    return a + b
}
```

Berkas: internal/calc/calc_test.go

```
package calc

import "testing"

func TestAdd(t *testing.T) {
    if got := Add(2, 3); got != 5 {
        t.Fatalf("Add(2, 3) = %d, want 5", got)
    }
}
```

> **CEK** Test contoh harus lulus.
>
> **`go test ./...`**
>
> **Hasil yang benar:** ok githooks-demo/internal/calc 0.00xs

Tahap 3. Kerangka program githook dan perintah install

Tujuan: Satu program Go yang akan menjalankan semua hook, plus perintah untuk memasangnya.

Kita menulis **satu program** bernama `githook`. Nama hook (pre-commit, commit-msg, pre-push) diberikan sebagai argumen. Prinsip yang perlu diingat: **kode keluar 0 berarti lanjut, selain 0 berarti Git membatalkan aksinya**.

Perintah `install` membuat folder `.githooks` berisi skrip pendek (shim) yang memanggil program Go ini, lalu mengatur `core.hooksPath` agar Git memakai folder tersebut. Karena `.githooks` ikut di-commit, seluruh tim memakai hook yang sama.

Berkas: cmd/githook/main.go

```
// Command githook menjalankan semua Git hooks kita.
package main

import (
    "bytes"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "strings"
)

const hooksDir = ".githooks"

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "pemakaian: githook <install|nama-hook>")
        os.Exit(2)
    }

    var err error
    switch os.Args[1] {
    case "install":
        err = install()
    default:
        err = fmt.Errorf("hook tidak dikenal: %q", os.Args[1])
    }

    if err != nil {
        fmt.Fprintf(os.Stderr, "\nGAGAL: %v\n", err)
        os.Exit(1) // kode keluar bukan 0 = Git membatalkan aksinya
    }
}

// install membuat folder .githooks berisi shim untuk tiap hook,
// lalu mengarahkan Git ke folder itu lewat core.hooksPath.
func install() error {
    if err := os.MkdirAll(hooksDir, 0o755); err != nil {
        return err
    }
    for _, name := range []string{"pre-commit", "commit-msg", "pre-push"} {
        // "$@" meneruskan argumen dari Git (mis. path file pesan commit).
        script := "#!/bin/sh\nexec go run ./cmd/githook " + name + " \"$@\"\n"
        path := filepath.Join(hooksDir, name)
        if err := os.WriteFile(path, []byte(script), 0o755); err != nil {
            return err
        }
        fmt.Println("[ok] dibuat", path)
    }

    out, err := exec.Command("git", "config", "core.hooksPath", hooksDir).CombinedOutput()
    if err != nil {
        return fmt.Errorf("git config gagal: %v\n%s", err, out)
    }
    fmt.Println("[ok] core.hooksPath =", hooksDir)
    return nil
}

// run menjalankan perintah dan mengembalikan stdout+stderr gabungan.
func run(name string, args ...string) (string, error) {
    cmd := exec.Command(name, args...)
    var buf bytes.Buffer
    cmd.Stdout, cmd.Stderr = &buf, &buf
    if err := cmd.Run(); err != nil {
        cmdline := name + " " + strings.Join(args, " ")
        return buf.String(), fmt.Errorf("%s gagal:\n%s", cmdline, buf.String())
    }
    return buf.String(), nil
```

```
}
```

Rapikan format kode (kode yang disalin dari PDF memakai spasi, sedangkan Go memakai tab):

Terminal

```
gofmt -w .
```

> **Perhatian.** Setelah `install` dijalankan di bawah, Git akan memanggil hook yang belum lengkap. **Jangan melakukan commit dulu** sampai Tahap 7. Jika terpaksa, lewati hook dengan `git commit --no-verify`.

Jalankan build lalu pasang hook:

Terminal

```
go build ./...
go run ./cmd/githook install
```

> **CEK** Hook terpasang dan Git diarahkan ke .githooks.
>
> ```
> go run ./cmd/githook install
> git config core.hooksPath
> cat .githooks/pre-commit
> ```
>
> **Hasil yang benar:** Install menampilkan 4 baris [ok]
> core.hooksPath menampilkan: .githooks
> Isi pre-commit: baris #!/bin/sh lalu: exec go run ./cmd/githook pre-commit "$@"

Tahap 4. Hook pre-commit: format dan go vet

Tujuan: Menolak commit jika file .go belum rapi atau go vet menemukan masalah.

Hook ini mengambil daftar file `.go` yang sudah di-stage, menjalankan `gofmt -l` (hanya mencetak nama file yang belum rapi), lalu `go vet`. Jika ada yang gagal, hook mengembalikan error sehingga commit batal.

Berkas: cmd/githook/precommit.go

```
package main

import (
    "fmt"
    "strings"
)

// preCommit memeriksa format (gofmt) file .go yang di-stage, lalu go vet.
func preCommit() error {
    fmt.Println("-> pre-commit: memeriksa format dan vet...")

    out, err := run("git", "diff", "--cached", "--name-only", "--diff-filter=ACM")
    if err != nil {
        return err
    }
    var goFiles []string
    for _, f := range strings.Fields(out) {
        if strings.HasSuffix(f, ".go") {
            goFiles = append(goFiles, f)
        }
    }
    if len(goFiles) == 0 {
        fmt.Println("   tidak ada file .go yang di-stage, dilewati")
        return nil
    }

    unformatted, err := run("gofmt", append([]string{"-l"}, goFiles...)...)
    if err != nil {
        return err
    }
    if strings.TrimSpace(unformatted) != "" {
        return fmt.Errorf("file belum rapi, jalankan `gofmt -w`:\n%s", unformatted)
    }

    if _, err := run("go", "vet", "./..."); err != nil {
        return err
    }
    fmt.Println("[ok] pre-commit lolos")
    return nil
}
```

Daftarkan hook ini di `cmd/githook/main.go`. Tambahkan dua baris `case` berikut **tepat di atas baris** `default:` di dalam `switch`:

Tambahkan ke cmd/githook/main.go

```
    case "pre-commit":
        err = preCommit()
```

Terminal

```
gofmt -w .
git add -A
go run ./cmd/githook pre-commit
```

> **CEK** Semua file sudah rapi, jadi hook harus lolos.
>
> **`go run ./cmd/githook pre-commit`**
>
> **Hasil yang benar:** -> pre-commit: memeriksa format dan vet...
> [ok] pre-commit lolos

Tahap 5. Hook commit-msg: Conventional Commits

Tujuan: Menolak pesan commit yang tidak mengikuti format type(scope): deskripsi.

Git memberi hook ini **path sebuah file** yang berisi pesan commit. Program membaca baris pertamanya dan mencocokkannya dengan pola Conventional Commits. Pesan merge dan revert otomatis dari Git dilewati.

Berkas: cmd/githook/commitmsg.go

```
package main

import (
    "fmt"
    "os"
    "regexp"
    "strings"
)

// Format Conventional Commits: type(scope)!: deskripsi
var conventional = regexp.MustCompile(
    `^(feat|fix|docs|style|refactor|perf|test|build|ci|chore|revert)` +
        `(\([a-z0-9._-]+\))?(!)?: \S.{0,71}$`)

// commitMsg membaca file pesan commit (path diberikan Git) dan memeriksa baris pertamanya.
func commitMsg(file string) error {
    data, err := os.ReadFile(file)
    if err != nil {
        return err
    }
    first := strings.SplitN(strings.TrimSpace(string(data)), "\n", 2)[0]

    // Pesan merge dan revert otomatis dari Git tidak perlu diperiksa.
    if strings.HasPrefix(first, "Merge ") || strings.HasPrefix(first, "Revert ") {
        return nil
    }
    if !conventional.MatchString(first) {
        return fmt.Errorf("pesan commit tidak sesuai Conventional Commits:\n  %q\n\n"+
            "format : <type>(scope opsional): <deskripsi>\n"+
            "contoh : feat(auth): tambah login dengan Google\n"+
            "type   : feat fix docs style refactor perf test build ci chore revert", first)
    }
    fmt.Println("[ok] commit-msg lolos")
    return nil
}
```

Tambahkan `case` berikut di `cmd/githook/main.go`, **tepat di atas baris** `default:` (di bawah case pre-commit):

Tambahkan ke cmd/githook/main.go

```
    case "commit-msg":
        if len(os.Args) < 3 {
            err = fmt.Errorf("commit-msg butuh path file pesan commit")
        } else {
            err = commitMsg(os.Args[2])
        }
```

Terminal

```
gofmt -w .
```

Uji langsung dengan berkas pesan sementara:

Terminal

```
echo "tambah fitur" > pesan.txt
go run ./cmd/githook commit-msg pesan.txt

echo "feat(app): tambah fitur" > pesan.txt
go run ./cmd/githook commit-msg pesan.txt

rm pesan.txt
```

> **CEK** Pesan pertama ditolak, pesan kedua lolos.
>
> `go run ./cmd/githook commit-msg pesan.txt`
>
> **Hasil yang benar:** Pesan pertama: GAGAL: pesan commit tidak sesuai Conventional Commits ... disertai format dan contoh
> Pesan kedua: [ok] commit-msg lolos

Tahap 6. Hook pre-push: jalankan test

Tujuan: Membatalkan push jika ada test yang gagal.

Hook ini paling sederhana: jalankan `go test ./...`, dan batalkan push bila gagal. Boleh lebih lambat daripada pre-commit karena jarang dijalankan.

Berkas: cmd/githook/prepush.go

```
package main

import (
    "errors"
    "fmt"
)

// prePush menjalankan seluruh test sebelum kode dikirim ke remote.
func prePush() error {
    fmt.Println("-> pre-push: menjalankan test...")
    out, err := run("go", "test", "./...")
    fmt.Print(out)
    if err != nil {
        return errors.New("test gagal, push dibatalkan")
    }
    fmt.Println("[ok] pre-push lolos")
    return nil
}
```

Tambahkan `case` terakhir di `cmd/githook/main.go`, **tepat di atas baris** `default:`:

Tambahkan ke cmd/githook/main.go

```
    case "pre-push":
        err = prePush()
```

Terminal

```
gofmt -w .
go vet ./...
go run ./cmd/githook pre-push
```

> **CEK** Program lengkap, rapi, dan test lulus.
>
> `gofmt -l .`
> `go run ./cmd/githook pre-push`
>
> **Hasil yang benar:** gofmt -l tidak mencetak apa pun
> pre-push menampilkan hasil test lalu: [ok] pre-push lolos

Tahap 7. Uji end-to-end dengan Git sungguhan

Tujuan: Melihat hook bekerja saat memakai git commit dan git push yang asli.

7.1 Commit pertama (harus lolos)

Terminal

```
git add -A
git commit -m "chore: inisialisasi proyek"
```

CEK Dua hook berjalan lalu commit dibuat.

```
git log --oneline
```

Hasil yang benar: [ok] pre-commit lolos
[ok] commit-msg lolos
satu baris riwayat: chore: inisialisasi proyek

7.2 Format berantakan (harus ditolak oleh pre-commit)

Buat berkas `internal/calc/sub.go` dengan isi berikut. **Sengaja berantakan, jangan dirapikan dulu.**

Berkas: internal/calc/sub.go

```
package calc
func   Sub(a,b int) int {return a-b}
```

Terminal

```
git add -A
git commit -m "feat: tambah sub"
```

CEK Commit dibatalkan dan hook menyebut nama berkasnya.

```
git log --oneline
```

Hasil yang benar: GAGAL: file belum rapi, jalankan `gofmt -w`: internal/calc/sub.go
Riwayat masih satu baris (commit tidak terbentuk)

7.3 Pesan asal-asalan (harus ditolak oleh commit-msg)

Terminal

```
gofmt -w internal/calc/sub.go
git add -A
git commit -m "tambah fungsi sub"
```

CEK pre-commit lolos, tetapi commit-msg menolak pesannya.

```
git log --oneline
```

Hasil yang benar: GAGAL: pesan commit tidak sesuai Conventional Commits

7.4 Pesan benar (harus lolos)

Terminal

```
git commit -m "feat(calc): tambah fungsi sub"
```

CEK Commit terbentuk.

```
git log --oneline
```

Hasil yang benar: dua baris riwayat; yang teratas: feat(calc): tambah fungsi sub

7.5 Push dengan test gagal (harus ditolak oleh pre-push)

Agar bisa mencoba `git push` tanpa GitHub, buat remote lokal (folder kosong bergaya server):

Terminal

```
git init --bare ../remote.git
git remote add origin ../remote.git
git push -u origin main
```

> **Catatan.** Pada remote lokal, Git kadang mencetak peringatan `push negotiation failed; proceeding anyway`. Ini tidak masalah: push tetap berhasil (terlihat dari baris `new branch main -> main`).

Push pertama ini harus lolos (test hijau). Sekarang rusak test dengan sengaja: ubah **dua** angka `5` di `internal/calc/calc_test.go` menjadi `6`, lalu:

Terminal

```
git commit -am "test(calc): ubah ekspektasi"
git push
```

> **CEK** Commit lolos (formatnya rapi) tetapi push dibatalkan.
>
> `git push`
>
> **Hasil yang benar:** FAIL: TestAdd ... Add(2, 3) = 5, want 6
> GAGAL: test gagal, push dibatalkan

Batalkan commit tadi agar test kembali hijau, lalu push lagi:

Terminal

```
git reset --hard HEAD~1
git push
```

7.6 Melewati hook (dan kenapa CI tetap perlu)

Terminal

```
git commit --allow-empty --no-verify -m "asal-asalan"
```

> **CEK** Commit terbentuk tanpa pemeriksaan apa pun.
>
> `git log --oneline`
>
> **Hasil yang benar:** baris teratas: asal-asalan (hook dilewati)

> **Catatan.** Hook lokal adalah alat kenyamanan, bukan penjaga keamanan: siapa pun bisa melewatinya dengan `--no-verify`. Itu sebabnya Tahap 9 mengulang pemeriksaan di CI. Hapus commit contoh tadi dengan `git reset --hard HEAD~1`.

Tahap 8. Opsional: pakai Lefthook untuk tim

Tujuan: Alternatif tanpa skrip install buatan sendiri.

Untuk tim, **Lefthook** (hook manager yang juga ditulis dengan Go, satu binary tanpa dependensi) lebih praktis. Konfigurasinya satu berkas YAML yang ikut di-commit.

Terminal

```
go install github.com/evilmartians/lefthook@latest
```

> **Perhatian.** Lefthook memasang hook ke folder bawaan Git, sedangkan `core.hooksPath` buatan kita mengarahkan Git ke folder lain. Agar tidak bentrok, lepas dulu pengaturan kita dengan `git config --unset core.hooksPath`. Pastikan folder bin Go (`go env GOPATH`, lalu subfolder bin) ada di PATH.

Berkas: lefthook.yml

```
pre-commit:
  commands:
    gofmt:
      glob: "*.go"
      run: test -z "$(gofmt -l {staged_files})" || (echo "jalankan gofmt -w" && exit 1)
    vet:
      glob: "*.go"
      run: go vet ./...

pre-push:
  commands:
    test:
      run: go test ./...
```

Terminal

```
git config --unset core.hooksPath
lefthook install
```

> **Catatan.** Tahap opsional ini ditulis berdasarkan dokumentasi Lefthook dan belum diuji langsung oleh penulis tutorial. Jika hasilnya berbeda, ikuti dokumentasi di github.com/evilmartians/lefthook.

> **CEK** Lefthook memasang hook ke .git/hooks.
>
> ```
> git add -A
> git commit -m "feat: coba lefthook"
> ```
>
> **Hasil yang benar:** Lefthook menampilkan blok pre-commit dengan hasil gofmt dan vet

Tahap 9. Mengulang pemeriksaan di CI (GitHub Actions)

Tujuan: Penjaga terakhir yang tidak bisa dilewati dari laptop.

Buat berkas workflow berikut. Setiap push ke `main` dan setiap pull request akan menjalankan pemeriksaan yang sama di server GitHub.

Berkas: .github/workflows/ci.yml

```
name: ci

on:
  push:
    branches: [main]
  pull_request:

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: stable
      - name: Cek format
        run: test -z "$(gofmt -l .)"
      - run: go vet ./...
      - run: go test ./...
```

Setelah repo Anda di-push ke GitHub, buka tab **Actions** di halaman repo.

> **CEK** Workflow `ci` muncul dan berjalan.
>
> **`(di browser) tab Actions pada repo GitHub`**
>
> **Hasil yang benar:** Satu run bernama ci dengan langkah gofmt, vet, dan test; centang hijau jika semuanya lulus

Pemecahan masalah

Gejala	Yang perlu dicek
Hook tidak jalan sama sekali	Cek `git config core.hooksPath` (harus `.githooks`). Di macOS/Linux pastikan executable: `chmod +x .githooks/*`. Git mengabaikan hook tanpa izin executable.
`hook tidak dikenal`	Case di `switch` pada main.go belum ditambahkan, atau salah ketik nama. Nama case harus persis sama dengan nama hook.
`go: command not found` saat commit dari aplikasi GUI	Aplikasi GUI kadang tidak membawa PATH terminal. Commit dari terminal, atau gunakan path penuh ke go di shim.
pre-commit menolak file yang menurut Anda rapi	Jalankan `gofmt -l .` untuk melihat berkas mana, lalu `gofmt -w .`. Kode hasil salin dari PDF memakai spasi, bukan tab.
Hook terasa lambat	`go run` mengompilasi tiap kali. Build sekali (`go build -o .githooks/bin/githook ./cmd/githook`) lalu ubah shim memanggil binary itu, atau pindahkan pemeriksaan berat ke pre-push dan CI.
Ingin melewati hook sesekali	`git commit --no-verify` atau `git push --no-verify`. Gunakan seperlunya; CI tetap akan memeriksa.

Langkah berikutnya

Tambahkan aturan sendiri, misalnya larangan commit langsung ke `main`, pemeriksaan `golangci-lint`, atau hook `post-merge` yang menjalankan `go mod tidy`. Untuk bahan bacaan:

git-scm.com/docs/githooks (dokumentasi resmi semua hook)

git-scm.com/book/en/v2/Customizing-Git-Git-Hooks (Pro Git, bab Git Hooks)

conventionalcommits.org/en/v1.0.0 (spesifikasi pesan commit)

github.com/evilmartians/lefthook (hook manager berbasis Go)

go.dev/tour (belajar Go dari nol)